Back propagation.

- core algorithm behind how neural networks learn

- learning = which weights & biases minimize a cost function.

- cost of single trainy example = diff b/w real output & desired output.

- looking for negative gradient of this cost function. tells you how to nudge w&b's to most quickly decrease the cost.

- backpropagation computes the crazy complicated gradient.

- magnitude of each how sensitive cost function is to each weight & bias.

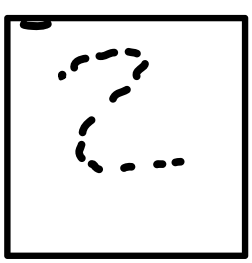
$-\nabla C(\ldots)$ ← all weights & biases

- backpropagation notation is hard

- will step through effect each training example is having on the weights & biases

- cost( ) involves averaging cost per example over all 10k+ training examples, way we adjust w & b's for a single ∇ descent step also depends on every single example

- focus on one training example:



cost of one training example

how should this one ex affect how w & b get adjusted.

- lets say activations in output are pretty random, untrained network

- we can only change weights & biases not the activations.

- focus on neuron whose activation we want to increase.

(0.2) but should be (1).

$$0.2 = \sigma(a_0 w_0 + a_1 w_1 + \dots + w_n a_n + b)$$

- three ways to increase activation:
  - → inc b
  - → inc w in proportion to activation
  - → change activations from prev layer

- weights have differing levels of influence
- gradient desent cares ab which adjustments give you most bang for your buck
- Hebbian theory: neurons that fire together wire together

- if everything connected to our neuron (that we wanted to increase) w/ a positive weight got brighter, & everything that connected w/ a negative weight got dimmer.

⇒ result is our neuron gets brighter like we want.

- get most bang for your buck by seeing weight changes proportional to your weights

- we can't influence activations. only control on weights & biases.
- but we also want all other neurons in last layer to become less active
- each other output neurons have own thoughts
- desires of all other output neurons for what should happen to second to last layer.

- in proportion to weights, & how much each neuron needs to change.

- adding up desired effect =

  list of nudges you want to

  happen to second to last layer.

- recursively apply to weights & biases that determine those values.

- this is just how one training example wishes to nudge the weights & biases.

- go through back prop routine for every other training example

- record how each of them want to change weights & biases.

- average together desired changes.
  - this is the negative gradient of the cost function.

- takes long time to add up influence of every single training example, every gradient descent step.

- shuffle training data into mini batches. compute step using mini batch.

- you'd get a little step, not full $-\nabla C()$ which depends on all training data.

- not the best step downhill, but each step gives computational speed up.

= stochastic gradient descent.

- Back prop = algo for determining how single training ex wants to nudge weights & biases. Based on which cause most rapid decrease to the cost.

true gradient descent step wants that for all training examples & averaging desired changes

- So instead randomly divide into batches, do same for those
- go through each mini batch, make adjustments
- now you can implement backpropagation.

- You need a lot of training data